

基于 STM32 与 Modbus RTU 的模拟矿 井液压支架监控系统设计

摘要

针对煤矿综采工作面液压支架动作控制和状态监测的设计需求，本文设计并实现了一套基于 STM32F103VET6 与 Modbus RTU 现场总线通信的矿井液压支架监控系统。系统由 WPF 上位机、STM32 下位机、TB6600 步进电机驱动器、两路步进电机以及虚拟角度和压力传感器模型组成。上位机作为 Modbus RTU 主站，完成串口连接、设备轮询、实时数据显示、动作控制、报警判断和历史数据存储；STM32 作为 Modbus RTU 从站，完成协议解析、输入寄存器数据组织、线圈动作触发、保持寄存器目标位置接收、步进电机脉冲控制和虚拟传感器更新。

系统在功能上实现了液压支架升高、降低、前进、后退、角度边界保护、压力模型计算、压力报警和多节点监控界面。下位机软件采用 Core、Lib、App 三层结构，Core 保留 CubeMX 生成代码，Lib 放置通用步进驱动、按键驱动和 Modbus RTU 协议，App 放置液压支架业务控制和传感器模拟。通信协议采用功能码 04、05、06，其中输入寄存器 0 返回角度偏移编码，输入寄存器 1 和 2 返回压力值放大 100 倍后的整数。测试结果表明，系统能够稳定完成上下位机通信和电机动作控制，角度范围、压力变化、前进后退目标位置控制和报警阈值均满足设计要求。

关键词：STM32F103VET6；Modbus RTU；现场总线；液压支架；WPF 上位机；步进电机控制

Abstract

A mine hydraulic support monitoring and control system based on STM32F103VET6 and Modbus RTU fieldbus communication is designed and implemented in this project. The system consists of a WPF host computer, an STM32 lower controller, two TB6600 stepper motor drivers, two stepper motors, and a virtual sensor model for angle and pressure. The WPF host computer works as a Modbus RTU master and is responsible for serial communication, device polling, real-time display, manual control, alarm judgment, trend curves, and historical data storage. The STM32 controller works as a Modbus RTU slave and is responsible for protocol parsing, input register organization, coil-triggered motion control, holding-register target position reception, stepper pulse generation, and virtual sensor updating.

The system realizes hydraulic support raising, lowering, forward movement, backward movement, angle limit protection, pressure calculation, pressure alarm, and multi-node monitoring interface functions. The lower-computer firmware is organized into Core, Lib, and App layers. The Core layer keeps CubeMX-generated initialization code, the Lib layer contains general drivers and the Modbus RTU protocol, and the App layer contains hydraulic-support motion control and sensor simulation logic. Test results show that the system can complete host-controller communication and motor control stably, and the angle range, pressure model, target-position movement logic, and alarm thresholds meet the design requirements.

Key words: STM32F103VET6; Modbus RTU; fieldbus; hydraulic support; WPF host computer; stepper motor control

目录	
摘要	2
Abstract	3
1 概述.....	1
1.1 选题背景与意义.....	1
1.2 本人完成的主要工作	1
1.3 系统主要功能	2
2 系统总体设计方案	3
2.1 总体结构	3
2.2 技术路线论证	3
2.3 方案特点	4
3 现场总线通信方案设计	5
3.1 Modbus RTU 协议选择.....	5
3.2 寄存器与线圈映射.....	6
3.3 通信可靠性设计.....	7
4 硬件系统设计	8
4.1 MCU 与外设配置	8
4.2 步进电机与驱动器设计	8
4.3 原理图、PCB 与元器件	9
5 下位机软件设计	17
5.1 软件分层结构	17
5.2 步进电机驱动模块.....	17
5.3 支架动作控制模块.....	18
5.4 虚拟传感器与压力模型	19
6 上位机软件设计	22
6.1 WPF 软件架构	22
6.2 Modbus 主站通信设计	22
6.3 报警与历史数据设计.....	23
6.4 不同角度下的上位机显示	23

7 系统调试与测试	26
7.1 调试过程	26
7.2 测试结果	26
8 关键程序实现	28
8.1 下位机压力模型关键代码	28
8.2 前进后退目标位置控制关键代码	29
8.3 Modbus 寄存器与线圈映射关键代码	29
8.4 步进电机启动与停止关键代码	31
8.5 上位机传感器读取关键代码	32
8.6 上位机报警判断关键代码	33
9 总结与展望	34
9.1 目标达成分析	34
参考文献	36

1 概述

1.1 选题背景与意义

液压支架是煤矿综采工作面的关键支护装备，其主要作用是在采煤机割煤、刮板输送机移架和顶板压力变化过程中保持工作面空间稳定。随着煤矿生产逐步向自动化、少人化和智能化方向发展，液压支架的动作控制、姿态监测、压力监测和故障报警已经不再是单纯的机械问题，而是典型的机电一体化与工业通信综合应用问题。

在真实煤矿现场，液压支架通常需要采集立柱压力、推移状态、支架姿态、工作面位置等信息，并通过控制器或集控系统进行协调控制。本设计不直接接入真实压力传感器和液压执行机构，而是使用 STM32 下位机、步进电机和虚拟传感器模型搭建一个可调试、可观察、可演示的教学型系统。系统通过两个步进电机分别模拟液压支架的前进/后退和升高/降低，通过内部算法生成角度、压力 1 和压力 2 数据，再由 WPF 上位机通过 Modbus RTU 读取并显示。这样的方案保留了工业控制系统的基本结构，同时降低了真实液压系统带来的安全风险和实验复杂度。

1.2 本人完成的主要工作

本设计完成的主要工作包括：第一，确定以“基于 STM32 与 Modbus RTU 现场总线的矿井液压支架监控系统”为题目；第二，设计电路原理图，PCB Layout、元器件选型、组装焊接；第三，完成 STM32F103VET6 下位机工程配置，确定 USART1、TIM2 PWM、GPIO、按键、LED 和 TB6600 控制引脚；第四，完成步进电机驱动、按键测试、Modbus RTU 从站、支架动作层和虚拟传感器模型；第五，自主开发 WPF 上位机监控软件，并完成其与 STM32 下位机的协议适配，实现串口连接、数据读取、曲线显示、报警日志、历史数据存储和手动控制；第六，完成系统调试，重点解决前进后退方向切换、移动步长不生效、升降角度边界和压力不同步等问题；第七，整理原理图、PCB、BOM、流程图、测试记录和源程序，形成完整设计报告。

1.3 系统主要功能

系统主要功能包括：上位机能够选择串口并以 115200、8N1 参数连接下位机；上位机能够周期性读取从站地址为 1 的 STM32 节点输入寄存器；上位机能够显示当前角度、压力 1、压力 2 并绘制实时曲线；上位机能够根据压力 1 判断正常、警告和紧急状态；上位机能够发送升高、降低、前进、后退控制命令；下位机能够根据保持寄存器目标位置判断前进或后退方向，并按照位置差值换算移动电机脉冲；下位机能够限制升降角度在 -15° 到 $+15^{\circ}$ 范围内，边界处继续操作不再驱动电机；下位机能够根据角度统一计算压力，避免压力数据和角度状态不同步。

2 系统总体设计方案

2.1 总体结构

系统总体结构由上位机监控层、现场总线通信层、下位机控制层和执行/模拟层组成。WPF 上位机运行在 Windows 环境中，通过串口发送 Modbus RTU 请求帧；STM32F103VET6 作为下位机从站，接收并解析请求，按照功能码完成寄存器读写和线圈动作触发；TB6600 驱动器接收 STM32 输出的方向和脉冲信号，驱动两台步进电机转动；虚拟传感器模块在 STM32 内部维护角度和压力状态，并把数据组织为 Modbus 输入寄存器供上位机读取。

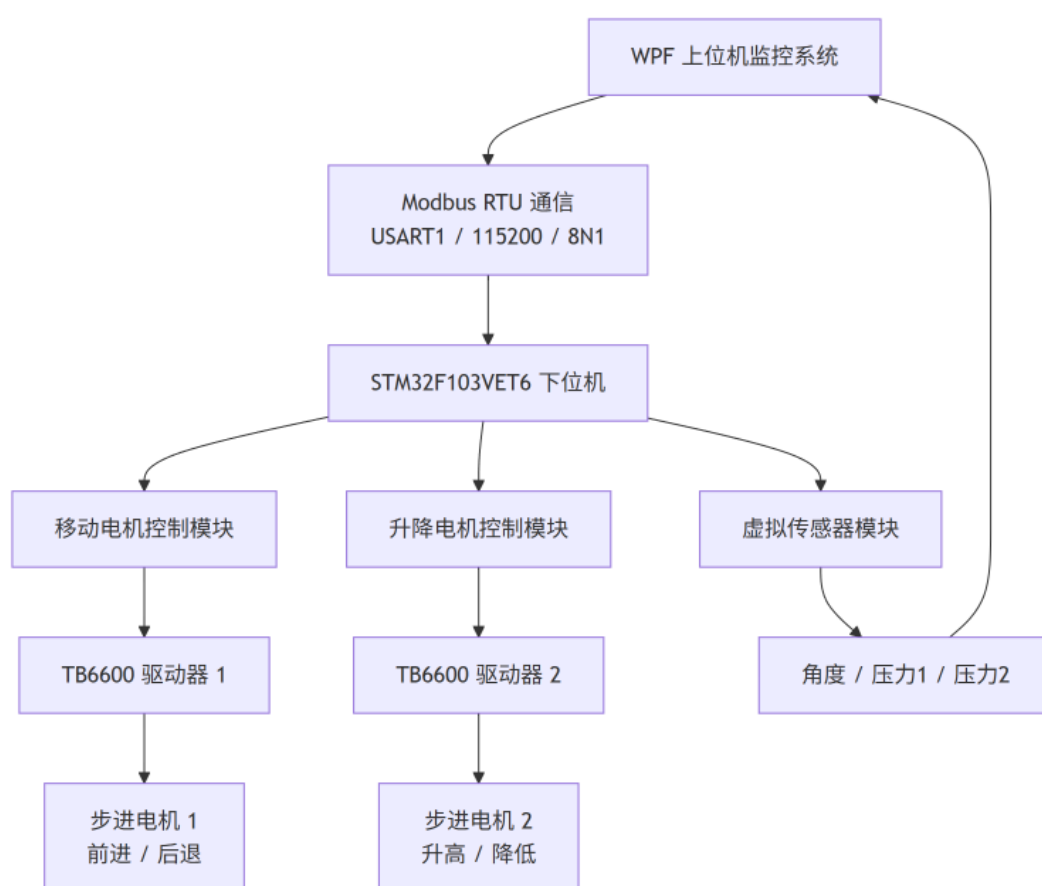


图 2-1 系统总体结构框图

2.2 技术路线论证

控制器选型方面，STM32F103VET6 具备 Cortex-M3 内核、较高主频、丰富 GPIO、定时器和串口资源，能够满足双路 PWM 脉冲输出、串口中断接收和业务控制逻辑的需要。电机执行方面，

TB6600 步进驱动器输入接口简单，适合通过 STEP/DIR 方式进行开环控制，便于在设计中直观观察支架动作。通信方面，Modbus RTU 是开放、成熟、易实现的工业通信协议，特别适合串口主从轮询和寄存器数据映射。本设计使用功能码 04、05、06 即可覆盖传感器数据读取、线圈动作触发和保持寄存器参数写入，协议复杂度适中。

上位机开发采用 WPF + VB.NET。WPF 具有较好的数据绑定和界面表现能力，结合 LiveCharts 能够显示实时趋势曲线；VB.NET 代码结构清晰，适合快速实现串口通信、数据库记录和界面控制。数据层采用 SQLite 保存历史采样数据，通信层使用 NModbus4 封装主站读写操作，并通过总线互斥锁避免轮询线程和人工控制指令同时占用串口。

2.3 方案特点

本设计的特点是软硬件边界清晰，通信协议与业务动作分离，既能体现硬件控制系统的设计过程，也能体现现场总线系统的工程应用过程。下位机没有把液压支架业务直接写入 Core 或 Lib 层，而是将底层协议、步进驱动和业务动作分层组织；上位机不直接计算压力和角度状态，而是以 STM32 返回的数据为准。这样设计可以降低上下位机状态不一致的风险，也便于后续扩展真实传感器或多支架节点。

3 现场总线通信方案设计

3.1 Modbus RTU 协议选择

Modbus 协议以客户端/服务器模型组织通信，在串行链路中常采用主站轮询从站的方式。它的数据模型包括线圈、离散输入、输入寄存器和保持寄存器等对象，适合把现场设备状态和控制参数抽象为统一地址空间。本设计中，WPF 上位机为主站，STM32 为从站。上位机主动发起读取或写入请求，STM32 收到请求后进行地址过滤、CRC 校验、功能码解析和响应帧返回。

Modbus RTU 帧采用二进制形式传输，帧内包含从站地址、功能码、数据区和 CRC 校验。相比自定义文本协议，Modbus RTU 具有更强的工程通用性和可调试性；相比复杂工业以太网协议，串口 RTU 更适合本设计的硬件规模和实现周期。

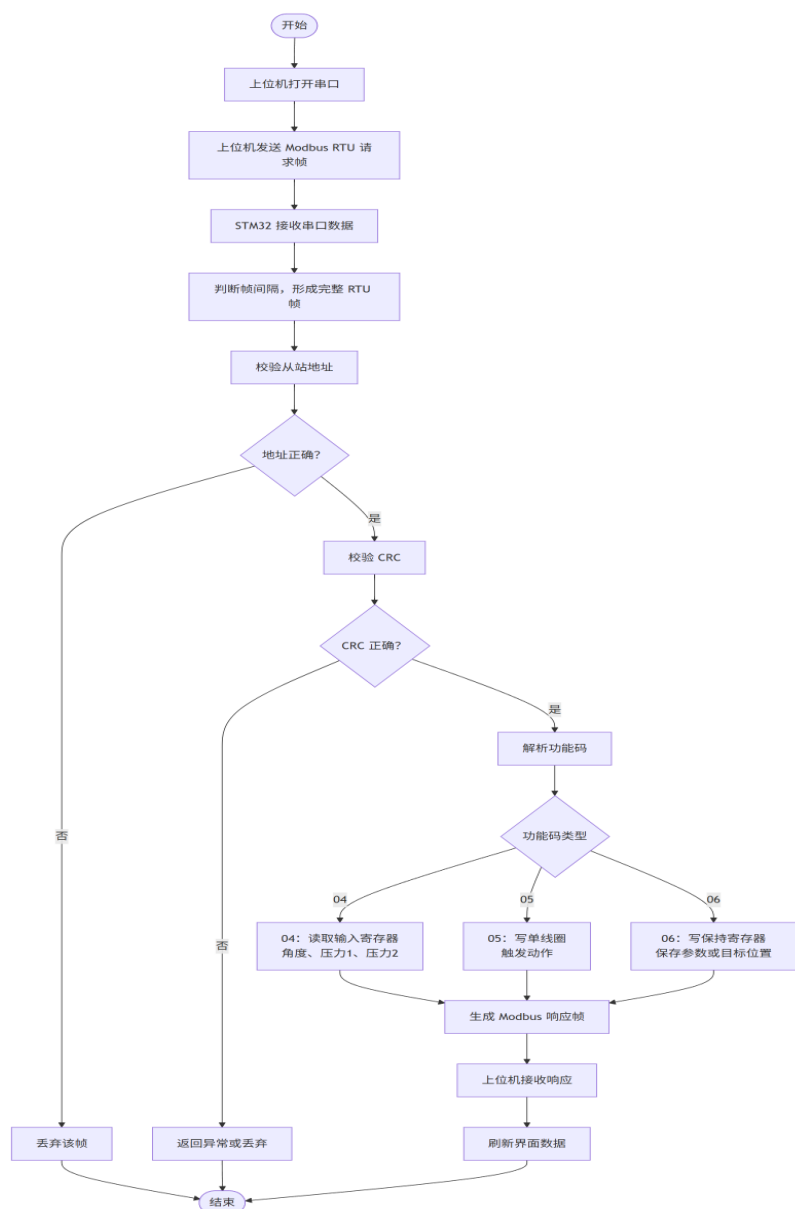


图 3-1 Modbus RTU 通信流程图

3.2 寄存器与线圈映射

地址类型	地址	含义	数据格式
输入寄存器	0	支架显示角度	actual_angle + 15
输入寄存器	1	压力 1	MPa × 100
输入寄存器	2	压力 2	MPa × 100
保持寄存器	1	下降参数	兼容保留
保持寄存器	2	上升参数	兼容保留
保持寄存器	3	前进/后退目标位置	uint16
线圈	0	下降触发	ON 触发
线圈	1	上升触发	ON 触发
线圈	2	移动触发	ON 触发

表 3-1 Modbus 地址映射表

角度寄存器采用偏移编码是因为 Modbus 输入寄存器本质上是 16 位无符号值，不能直接表达负角度。系统规定实际角度 -15° 对应寄存器值 0，实际角度 0° 对应寄存器值 15，实际角度 $+15^{\circ}$ 对应寄存器值 30。上位机读取后执行 registers(0)-15.0 还原为实际角度。压力寄存器采用放大 100 倍的整数表示方式，例如 26.00 MPa 对应寄存器值 2600，35.00 MPa 对应 3500。

3.3 通信可靠性设计

STM32 下位机在 Modbus RTU 层实现从站地址过滤、CRC16 校验、接收缓冲和 5ms 帧空闲边界判定。收到非法地址、CRC 错误或溢出帧时，系统不执行动作，避免错误数据引发误控制。上位机通信层通过 SyncLock 对串口总线加锁，使定时轮询和人工控制指令排队执行；同时将控制指令放入后台线程，避免界面阻塞。上位机写控制命令时先写保持寄存器，再延时 50ms 写线圈触发动作，以保证下位机先收到参数再执行动作。

4 硬件系统设计

4.1 MCU 与外设配置

下位机主控采用 STM32F103VET6。系统时钟配置为 HSE 8MHz，经 PLL 倍频到 72MHz。USART1 使用 PA9/PA10，参数为 115200、8 数据位、无校验、1 停止位，用于 Modbus RTU 通信。TIM2 的 CH1 和 CH2 分别输出移动电机和升降电机的 STEP 脉冲，方向信号由 GPIO 输出。LED 用于运行、通信和电机状态指示，按键用于脱离上位机时进行基本动作测试。

功能	引脚	说明
移动电机 STEP	PA0	TIM2 CH1 PWM 输出
升降电机 STEP	PA1	TIM2 CH2 PWM 输出
移动电机 DIR	PD12	方向控制
移动电机 ENA	PD13	使能预留
升降电机 DIR	PD14	方向控制
升降电机 ENA	PD15	使能预留
USART1 TX/RX	PA9/PA10	Modbus RTU 通信
按键	PE13/PE14/PE15	升高、降低、移动测试
LED	PB6/PB7/PB8	运行、电机、通信指示

表 4-1 STM32 主要引脚分配

4.2 步进电机与驱动器设计

系统使用两个 TB6600 驱动器和两个 42 步进电机。其中移动轴用于模拟支架前进和后退，升降轴用于模拟支架升高和降低。STM32 输出 STEP 脉冲决定电机转动步数，输出 DIR 电平决定电机方向。TB6600 的 PUL+ 和 DIR+ 接 3.3V，PUL- 和 DIR- 分别接 STM32 的 STEP 和 DIR 信号；电机电源由外部 12V 电源提供，不由 STM32 开发板供电。

在电机脉冲设计中，移动轴采用“位置差值到脉冲数”的映射方式：位置差 10 对应 80 个脉冲。上位机设置的 MoveStep 先转换为目标位置，下位机再比较 targetPosition 和 currentPosition，按差值计算实际脉冲数，因此不同步长会得到不同转动角度。升降轴采用累计目标脉冲映射，显示角度 -15° 到 +15° 对应升降电机 -3200 到 +3200 脉冲，从而避免每度约 213.33 脉冲造成的小数累计误差。

4.3 原理图、PCB 与元器件

硬件资料包含电路原理图、PCB 图和 BOM 清单。原理图体现 STM32 最小系统、晶振、复位、电源、串口、电机驱动信号、按键和 LED 等连接关系；PCB 图体现实际布线和器件布局；BOM 清单给出主要元器件型号、封装、数量和供应信息。

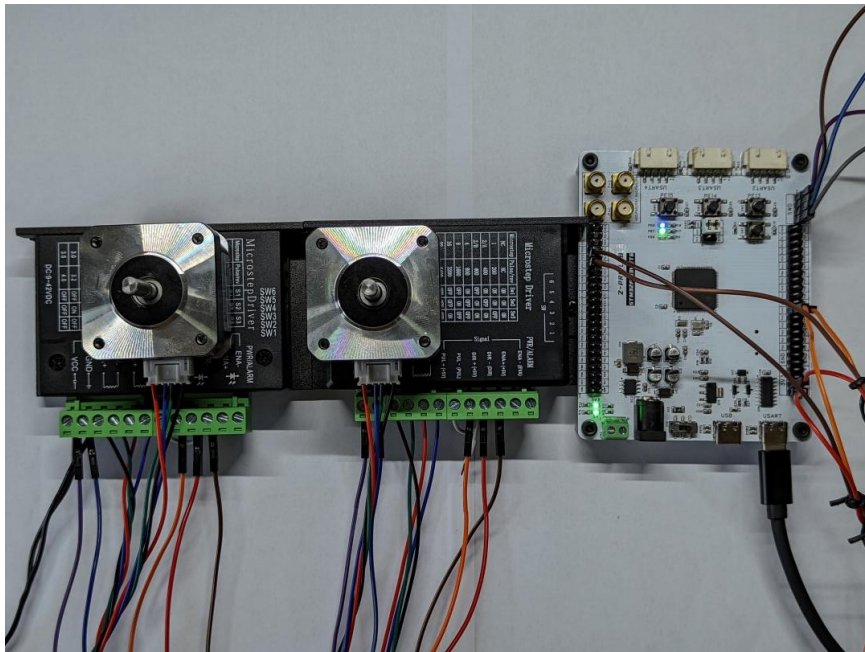


图 4-1 系统整体实物图

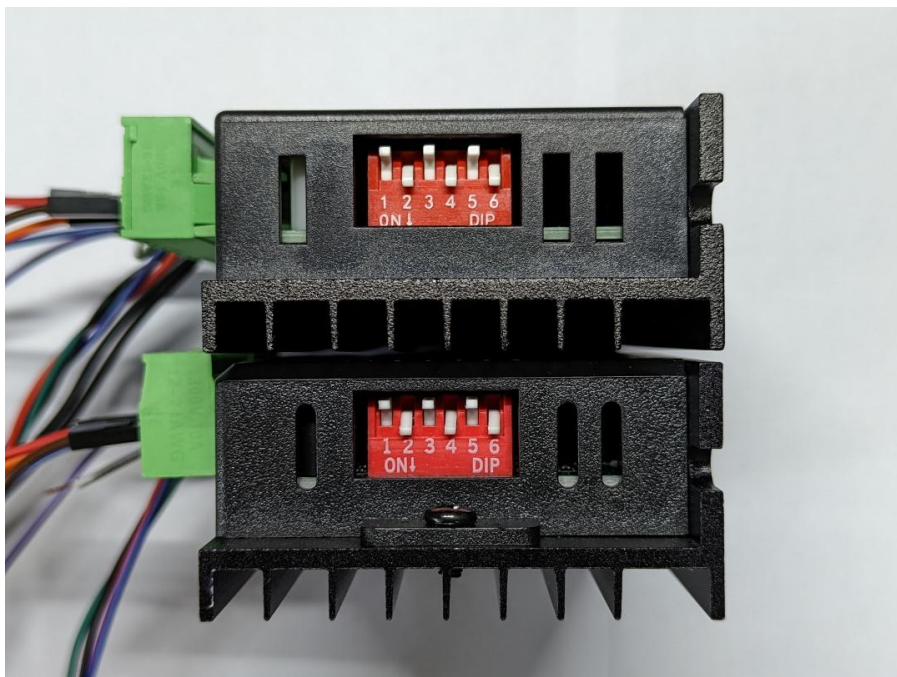


图 4-2 TB6600 驱动器与电机接线实物图

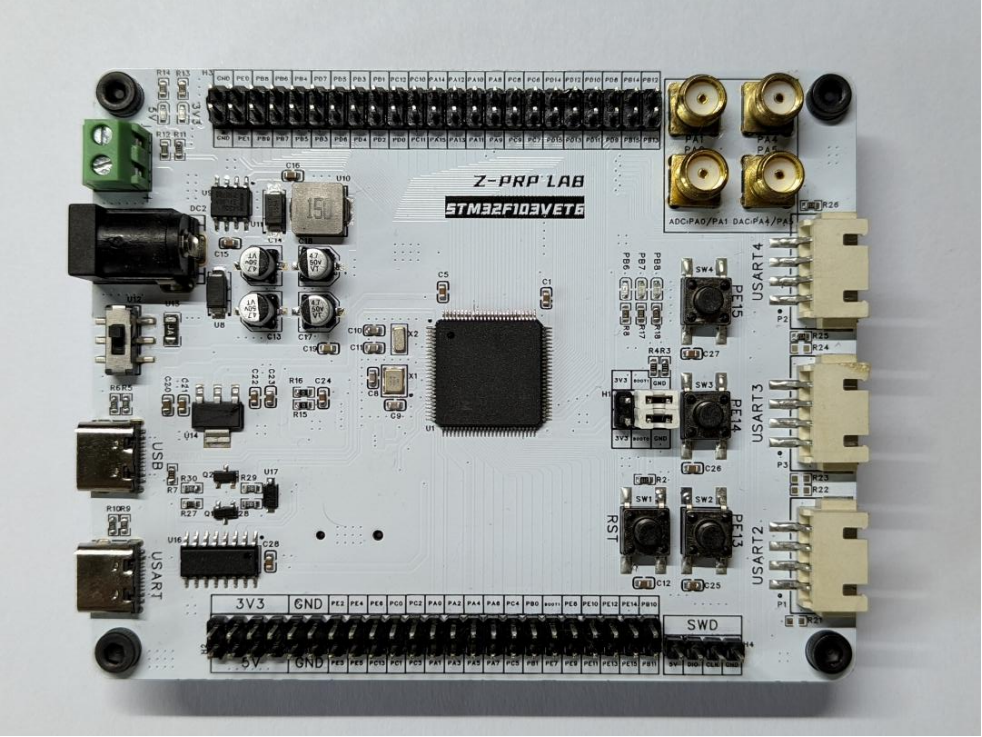


图 4-3 PCB 实物图

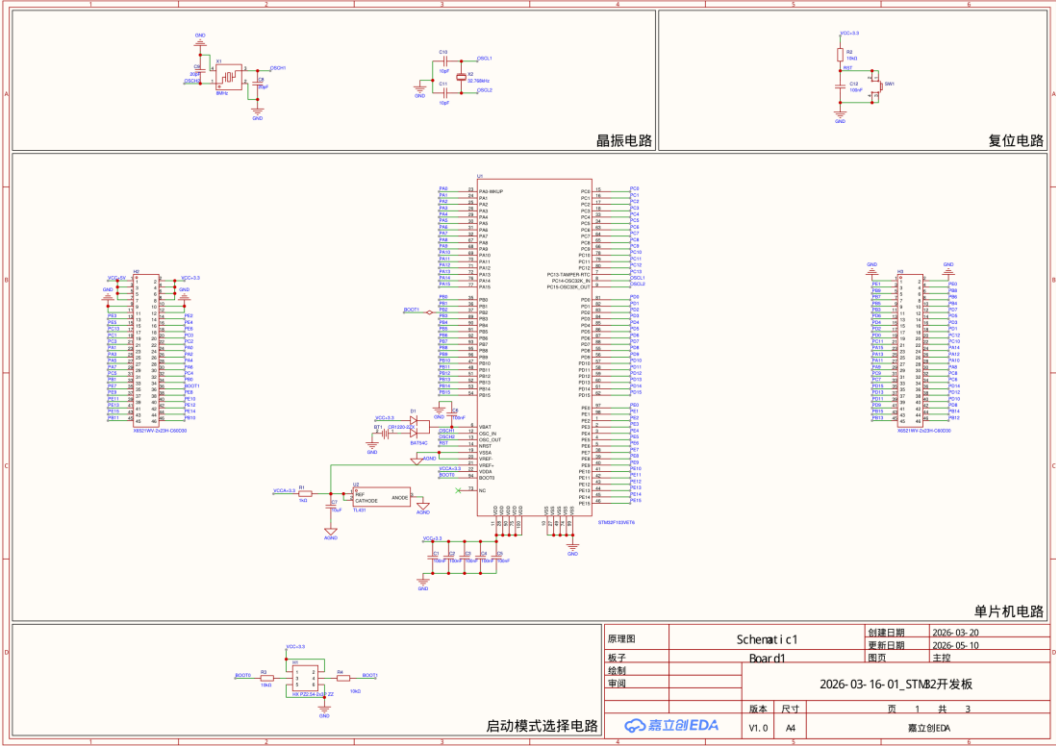


图 4-4 电路原理图第 1 页

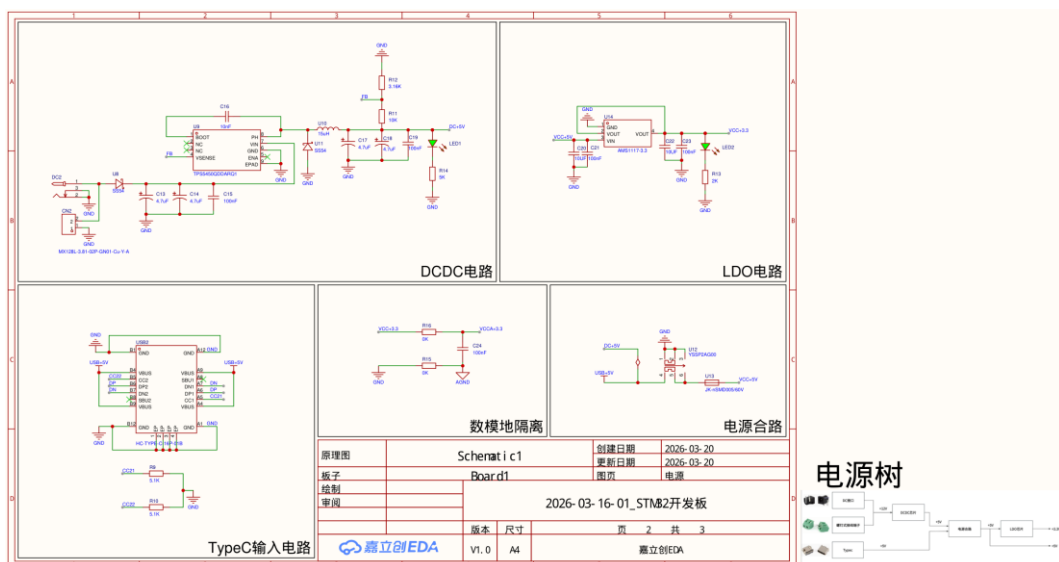


图 4-5 电路原理图第 2 页

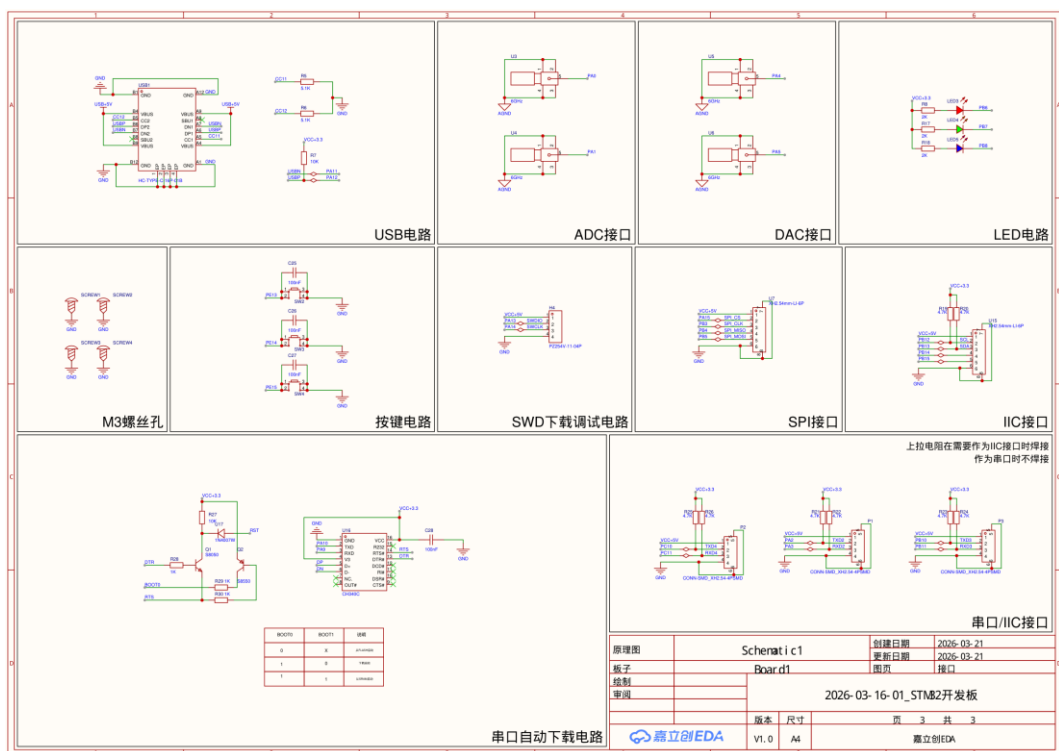


图 4-6 电路原理图第 3 页

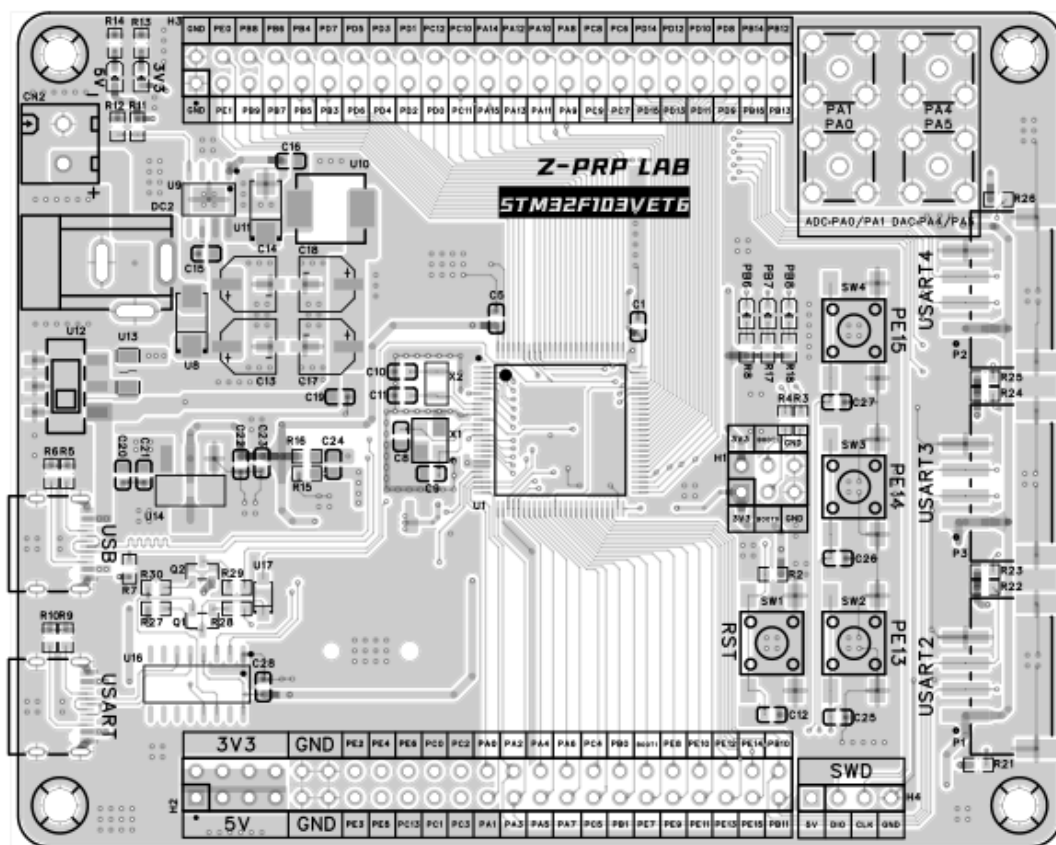


图 4-7 PCB Layout 第 1 页

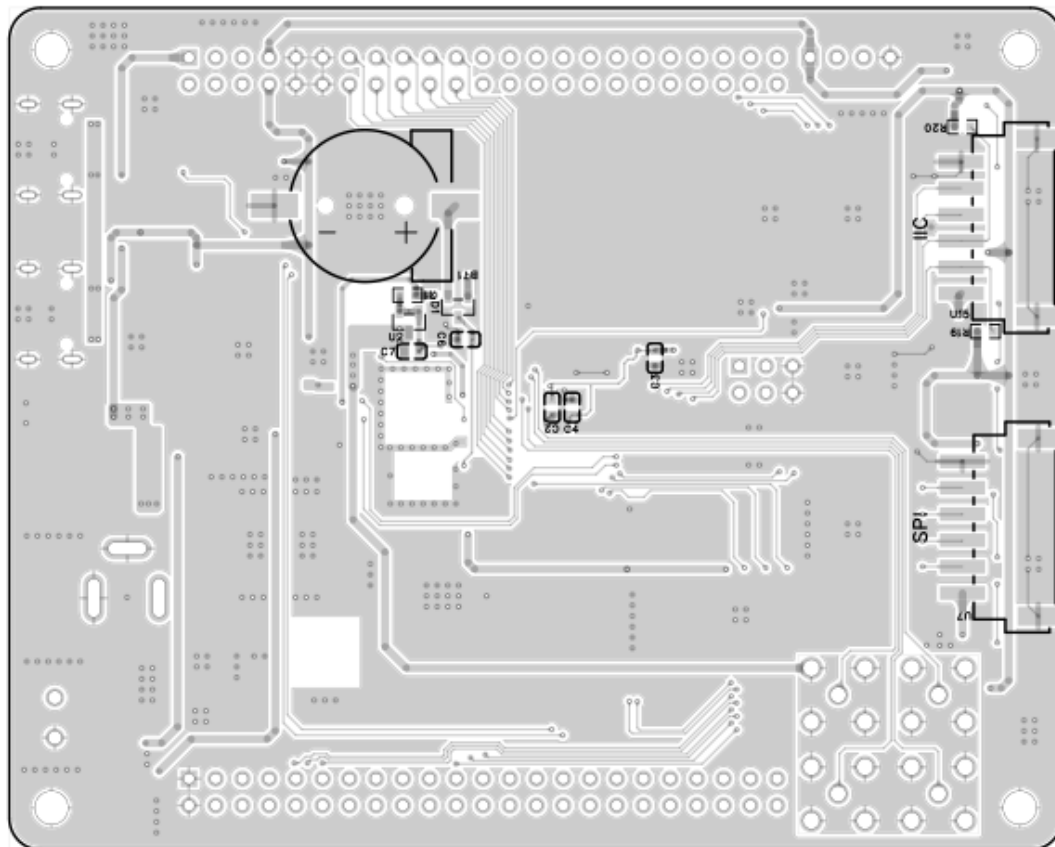


图 4-8 PCB Layout 第 2 页

N o .	Q t y	Comme nt	Designator	Footpr int	MPN	Manu factur er	Suppli er Part
电阻							
1	2	0Ω	R15,R16	R0603	0603WAF000T5E	UNI-ROYAL(厚声)	C21189
2	4	1kΩ	R1,R28,R29,R30	R0603	0603WAF1001T5E	UNI-ROYAL(厚声)	C21190
4	4	2kΩ	R8,R13,R17,R18	R0603	0603WAF2001T5E	UNI-ROYAL(厚声)	C22975
5	1	3.16kΩ	R12	R0603	RMC06033.16K1%N	Tyohm(幸亚电阻)	C325684
6	8	4.7kΩ	R19,R20,R21,R22,R23,R24,R25,R26	R0603	0603WAF4701T5E	UNI-ROYAL(厚声)	C23162
8	1	5kΩ	R14	R0603	RT0603BRE075KL	YAGEO(国巨)	C862232
9	4	5.1kΩ	R5,R6,R9,R10	R0603	0603WAF5101T5E	UNI-ROYAL(厚声)	C23186
12	6	10kΩ	R2,R3,R4,R7,R11,R27	R0603	0603WAF1002T5E	UNI-ROYAL(厚声)	C25804
电容							
14	2	10pF	C10,C11	C0603	CL10C100JB8NNNC	SAMSUNG(三星)	C1634
17	2	20pF	C8,C9	C0603	CL10C200JB8NNNC	SAMSUNG(三星)	C1648
13	1	10nF	C16	C0603	0603B103K500NT	FH(风华)	C57112
19	16	100nF	C1,C2,C3,C4,C5,C6,C12,C15,C19,C21,C23,C24,C25,C26,C27,C28	C0603	CGA0603X7R104K500JT	HRE(芯声)	C6119867
7	4	4.7uF	C13,C14,C17,C18	CAP-SMD_BD5.0-L5.3-W5.3-FD	EMVE500ADA4R7ME55G	NCC(日本贵弥功)	C178270
15	3	10uF	C7,C20,C22	C0603	CL10A106KP8NNNC	SAMSUNG(三星)	C19702
电源电感							
16	1	15uH	U10	IND-SMD_L6.6-W6.6	15uH		C9900014578
保险丝							
28	1	JK-nSMD005/60V	U13	F1206	JK-nSMD005/60V	JK(金科)	C2830235
晶振							

18	1	32.768k Hz	X2	OSC-SMD_L3.2-W1.5	SF32WK32768D31T002	TKD(泰晶)	C271654
11	1	8MHz	X1	CRYSTAL-SMD_4P-L3.2-W2.5-BL_SIT8008BI	XL2EL89CSI-111YLC-8M	YXC(扬兴晶振)	C5444548
二极管							
3	1	1N4007W	U17	SOD-123FL_L2.7-W1.8-LS3.7-RD	1N4007W	MSKSEMI(美森科)	C7528871
21	1	BAT54C	D1	SOT-23-3_L2.9-W1.3-P1.90-LS2.4-BR	BAT54C		C9900013520
38	2	SS54	U8,U11	SMA_L4.2-W2.6-LS5.1-RD	SS54		C9900023219
30	1	KT-0603R	LED3	LED_0603	KT-0603R	KENTO	C2286
31	3	LTST-C190KGKT	LED1,LED2,LED4	LED_0603	LTST-C190KGKT	LITEON(光宝)	C125094
32	1	LTST-C193TBKT-5A	LED5	LED_0603	LTST-C193TBKT-5A	LITEON(光宝)	C125103
三极管							
36	1	S8050	Q1	SOT-23-3_L2.9-W1.3-P1.90-LS2.4-BR	S8050	Doeshare(德芯)	C2931719
37	1	S8550	Q2	SOT-23-3_L2.9-W1.3-P1.90-LS2.4-BR	S8550	Doeshare(德芯)	C2931720
接口							
10	4	6GHz	U3,U4,U5,U6	SMA-TH_HL-SMA-KHD2	HL-SMA-KE-02	HenryTech(恒利泰)	C20617215
23	3	CONN-SMD_XH2.54-4PSMD	P1,P2,P3	CONN-SMD_XH2.54-4PSMD	CONN-SMD_XH2.54-4PSMD		C9900018341
26	2	HC-TYPE-C-16P-01B	USB1,USB2	USB-C-SMD_KH-TYPE-C-16P	HC-TYPE-C-16P-01B	HCTL(华灿天禄)	C2894898
27	1	HXPZ2.54-	H1	HDR-TH_6P-	HXPZ2.54-2x3P ZZ	hanxia(韩下)	C32713276

		2x3P ZZ		P2.54-V-R2-C3-S2.54_A 2541WV-2X3P			
35	1	PZ254V-11-04P	H4	HDR-TH_4P-P2.54-V-M	PZ254V-11-04P	XFCN(兴飞)	C2691448
43	2	XH2.54mm-LI-6P	U7,U15	CONN-SMD_6P-P2.50-H	XH2.54mm-LI-6P		C2842318
42	2	X6521WV-2x23H-C60D30	H2,H3	HDR-TH_46P-2.54-V-M-R2-C23-S2.54	X6521WV-2x23H-C60D30	XKB Connection(中国星坤)	C725895
芯片							
39	1	STM32F103VET6	U1	LQFP-100_L14.0-W14.0-P0.50-LS16.0-BL	STM32F103VET6	ST(意法半导体)	C8308
20	1	AMS1117-3.3	U14	SOT-223-3_L6.5-W3.4-P2.30-LS7.0-BR	AMS1117-3.3	AMS	C6186
22	1	CH340C	U16	SOP-16_L10.0-W3.9-P1.27-LS6.0-BL	CH340C	WCH(南京沁恒)	C84681
40	1	TL431	U2	SOT-23-3_L2.9-W1.3-P1.90-LS2.4-BR	TL431	YFSEMI(江苏佑风微)	C2887585
41	1	TPS5450QDDARQ1	U9	SOIC-8_L4.9-W3.9-P1.27-LS6.0-BL-EP	TPS5450QDDARQ1	TI(德州仪器)	C181270
供电							
24	1	CR1220-2ZX	BT1	BAT-SMD_CR1220-2ZX	CR1220-2ZX	QIJEY	C969906
25	1	DC005-2.5	DC2	DC-005_2.5	DC005-2.5	SHOUHAN(首尔)	C7507454
34	1	MX128L-3.81-02P-GN01-Cu-Y-A	CN2	CONN-TH_2P-P3.81_MX128L-3.81-GN01-02P-CU-Y-A	MX128L-3.81-02P-GN01-Cu-Y-A	MAX(迈旭)	C5188452
按键							
29	4	Key_SMD_6x6x6.5	SW1,SW2,SW3,SW4	SW-SMD_4P-L6.0-W6.0-P4.50-LS9.0-2			
4	1	YSSP2AG0	U12	SW-	YSSP2AG00	YIYUA	C518855

4		0		SMD_Y SSP2AG 00		N(怡远)	8
3 3	4	M3 螺丝	SCREW1,SCREW2,SCREW3,SCREW4	M3 螺丝			

表 4-2 完整 BOM 清单

5 下位机软件设计

5.1 软件分层结构

下位机软件采用 Core、Lib、App 三层结构。Core 层由 CubeMX 生成，主要包括 GPIO、TIM、USART、NVIC、系统时钟和中断入口；Lib 层放置通用底层模块，包括 stepper_driver、button_driver 和 modbus_rtu；App 层放置液压支架业务模块，包括 app、board_config、key_control、support_motion、sensor_simulator 和 modbus_app。这样的分层能保证 CubeMX 重新生成时不会破坏业务逻辑，也能避免底层协议或驱动代码包含液压支架专有概念。

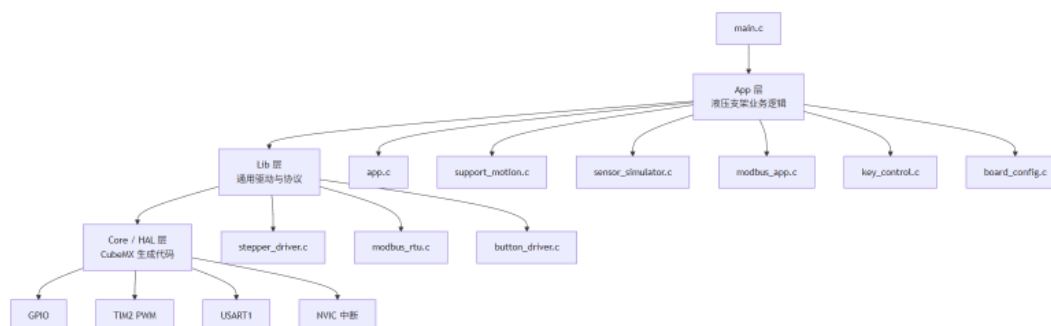


图 5-1 下位机软件分层结构图

5.2 步进电机驱动模块

stepper_driver 模块通过 TIM2 PWM 输出 STEP 脉冲，并在 HAL_TIM_PWM_PulseFinishedCallback 中统计脉冲完成数量。当 current_steps 达到 target_steps 后，驱动模块停止对应 PWM 通道，将 STEP 输出强制置为低电平，并将 ENA 写为非有效电平。模块提供 Stepper_MoveSteps、Stepper_Stop、Stepper_IsBusy 等接口。业务层只关心电机编号、方向、步数和频率，不直接访问 TIM 或 GPIO。

驱动模块还实现了 busy 保护。如果同一电机仍在运行，则再次启动返回 HAL_BUSY。由于两个电机共用 TIM2，不同通道在同一时刻若使用不同频率会产生定时器周期冲突，因此代码限制在已有电机忙碌时不能切换 active_frequency_hz。该设计保证了系统行

为可预测，也避免了快速连续按键或上位机重复点击造成脉冲计数混乱。

5.3 支架动作控制模块

`support_motion` 模块负责把“升高、降低、前进、后退、移动到目标位置”等业务动作转换为电机动作。升高和降低命令首先读取当前角度，判断是否到达 $+15^\circ$ 或 -15° 边界，未到边界时计算目标角度和目标累计脉冲，再启动升降电机。只有 `Stepper_MoveSteps` 返回 `HAL_OK` 后，系统才更新 `lift_pulse_position` 和虚拟角度，避免电机未启动而界面状态提前变化。

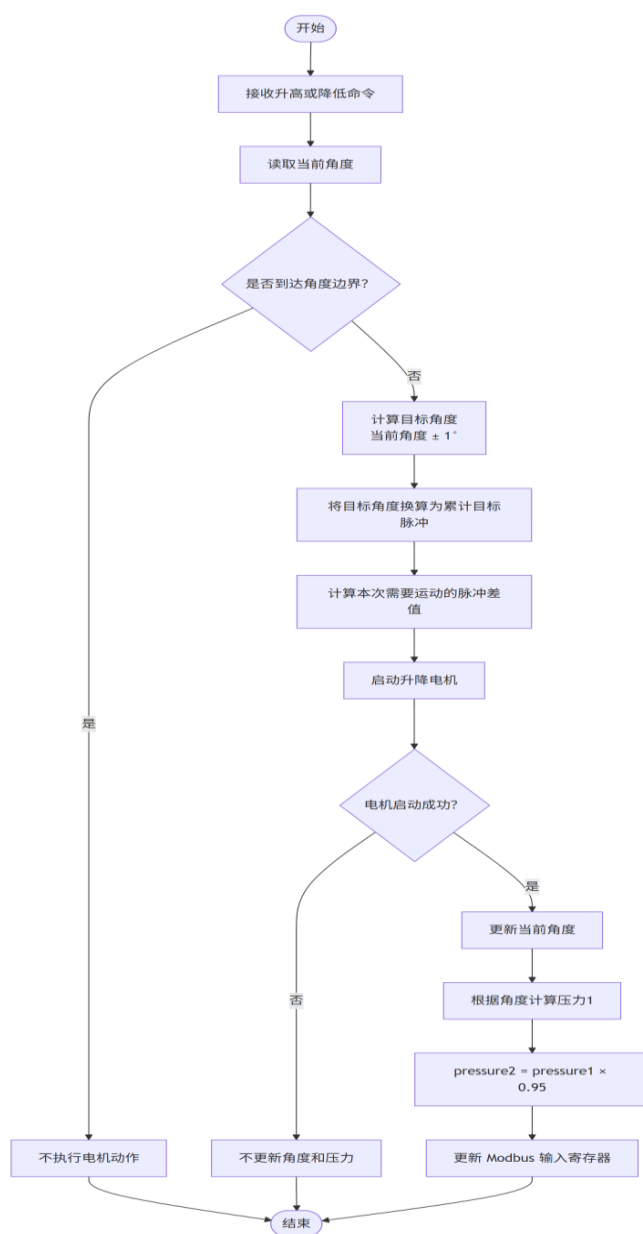


图 5-2 升降电机控制流程图

前进和后退采用目标位置控制。上位机并不直接发送“前进方向码”或“后退方向码”，而是写入保持寄存器 3 表示目标位置，再写线圈 2 触发移动。STM32 根据 `targetPosition` 与 `currentPosition` 的大小关系判断方向：目标位置大于当前位置则前进，小于当前位置则后退，相等则不动作。移动动作成功启动后，`currentPosition` 才更新为目标位置。

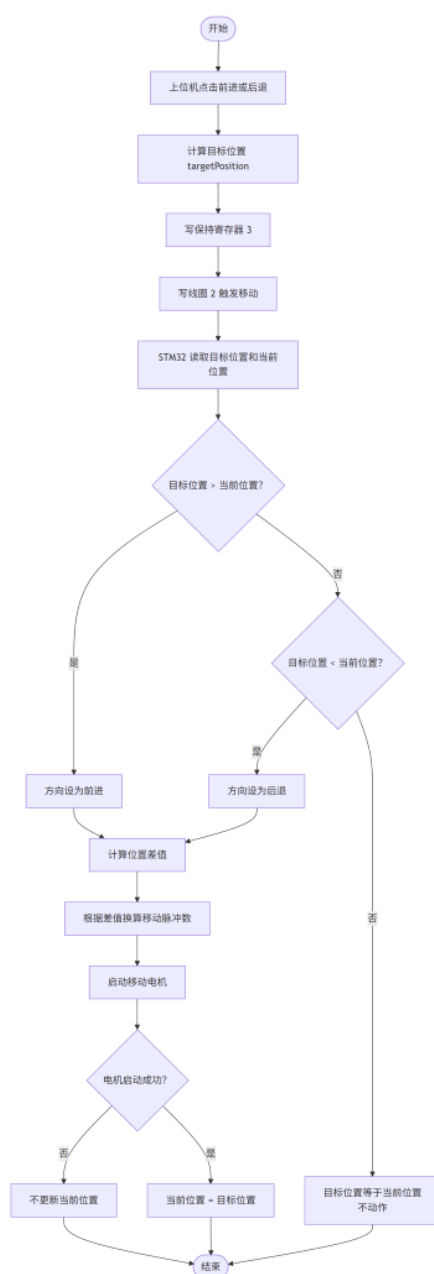


图 5-3 前进后退目标位置控制流程图

5.4 虚拟传感器与压力模型

`sensor_simulator` 模块维护当前显示角度、压力 1 和压力 2。角度范围为 -15° 到 $+15^{\circ}$ ，系统启动默认角度为 0° 。压力不再采用历史累加方式，而是由当前角度统一计算，前进和后退动作不独立改变压力。这样可以避免出现角度已经降到边界但压力仍有残余值的不同步现象。

压力模型规定： $\text{angle} \leq -13^{\circ}$ 时 `pressure1` 为 0 MPa； $-13^{\circ} < \text{angle} \leq 0^{\circ}$ 时 `pressure1` 由 0 MPa 线性增加到 26 MPa； $0^{\circ} < \text{angle} < +13^{\circ}$ 时 `pressure1` 由 26 MPa 线性增加到 35 MPa； $\text{angle} = +13^{\circ}$ 时进入紧急压力起点，`pressure1` 为 35 MPa； $\text{angle} = +14^{\circ}$ 时约为 37 MPa； $\text{angle} = +15^{\circ}$ 时为 40 MPa。`pressure2` 始终由 `pressure1` $\times 0.95$ 得到。

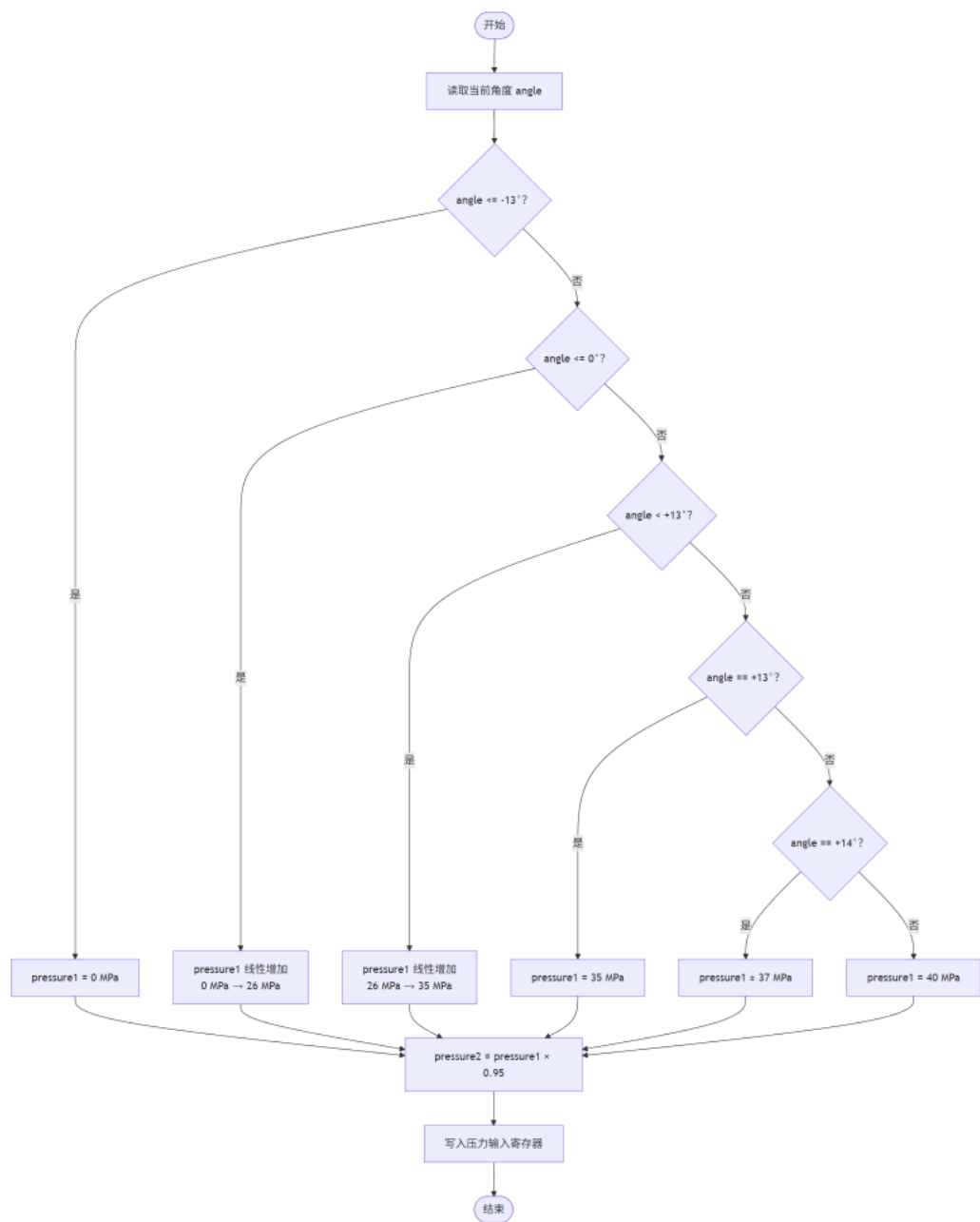


图 5-4 压力模型流程图

6 上位机软件设计

6.1 WPF 软件架构

上位机采用 WPF + VB.NET 开发，整体遵循 MVVM 思路。Models 目录包含 SupportModel 和 AlarmModel，分别描述支架节点状态和报警日志；ViewModels 目录中的 MainViewModel 是界面数据绑定、定时轮询、命令执行和报警判断的核心；Services 目录包含 ModbusService、DatabaseService 和历史保留的 SerialCommunicationService，其中当前主流程使用 ModbusService 进行 RTU 主站通信，使用 DatabaseService 将采样数据保存到 SQLite 数据库。

界面由 MainWindow.xaml 实现，左侧显示 56 个支架节点及在线状态，右侧显示当前压力、角度、实时趋势曲线、控制中心和报警日志。串口配置窗口允许选择端口号、波特率、数据位、停止位和校验位。系统默认波特率为 115200，数据位为 8，校验为 None，停止位为 1。



图 6-1 a 上位机主界面截图



图 6-1 b 上位机深色界面截图

6.2 Modbus 主站通信设计

ModbusService 中的 ReadSupportSensors 函数使用功能码 04 从指定从站读取输入寄存器 0 到 2。读取成功后，上位机将 registers(0)-15.0 还原为实际角度，将 registers(1)/100.0 和 registers(2)/100.0 还原为压力 1、压力 2。ControlSupport 函数用于发送控制命令，先通过功能码 06 写保持寄存器，再通过功能码 05 写

线圈触发动作。为了避免 RS485 半双工总线上的并发冲突，读写操作均放入 SyncLock 临界区。

6.3 报警与历史数据设计

报警判断以压力 1 为主。pressure1 $\geq$ 35.0 MPa 时触发紧急报警，pressure1 $\geq$ 30.0 MPa 且小于 35.0 MPa 时触发警告报警，pressure1 $<$ 30.0 MPa 时视为正常状态。pressure2 主要用于显示和趋势曲线，不作为主要报警判断依据。历史数据保存到 SQLite 数据库 SensorHistory 表，字段包括 SupportID、Angle、Pressure1、Pressure2 和 RecordTime。上位机切换支架节点时，会读取该节点最近 20 条历史数据并刷新图表。

6.4 不同角度下的上位机显示

为了验证压力模型与上位机显示的一致性，系统分别在 -13° 、 0° 、 8° 和 14° 等典型角度下进行显示测试。 -13° 对应压力接近 0 MPa， 0° 对应正常支护压力约 26 MPa， 8° 进入压力警告区间， 14° 进入紧急压力区间。不同角度截图能够证明上位机不是固定显示数据，而是能够正确解析 STM32 输入寄存器并随角度变化刷新压力、曲线和报警信息。



图 6-2 上位机角度 -13° 显示结果



图 6-3 上位机角度 0° 显示结果



图 6-4 上位机角度 8° 警告状态显示结果



7 系统调试与测试

7.1 调试过程

系统调试按照“单模块测试、下位机独立测试、通信联调、整体稳定性测试”的顺序进行。首先在按键测试模式下验证两个 TB6600 和两个步进电机的接线、方向和脉冲响应；然后启用 Modbus 模式，使用上位机连接 STM32，验证输入寄存器读取和线圈动作触发；最后进行连续动作、边界动作、压力模型、断开重连和快速误操作测试。

调试过程中曾出现前进切换后退后方向滞后的问题。分析后发现，上位机发送的是目标位置而不是方向码，旧版下位机每次只执行固定 80 脉冲动作，且上位机会提前更新界面位置，导致上下位机内部位置不同步。修正后，SupportMotion_MoveToPosition 按目标位置差值决定方向和脉冲数，并且只有电机启动成功才更新 currentPosition。

升降调试阶段曾出现电机实际转角与上位机显示角度不一致、到 0° 或 90° 后继续点击仍会转动的问题。修正后，角度范围改为 -15° 到 +15°，每次升降改变 1°，升降电机按累计目标脉冲映射，边界处直接返回，不再启动电机。压力模型也由独立加减改为根据角度统一计算，解决了压力 1 和压力 2 不同步的问题。

7.2 测试结果

测试项目	操作方法	预期结果	实际结果
串口连接	选择实际 COM 口，115200 8N1	上位机连接成功	通过
输入寄存器读取	读取地址 0~2	显示角度、压力 1、压力 2	通过
升高控制	点击升高按钮	角度 +1°，升降电机正向转动	通过
降低控制	点击降低按钮	角度 -1°，升降电机反向转动	通过
角度上限	达到 +15° 后继续升高	电机不再动作	通过
角度下限	达到 -15° 后继续降低	电机不再动作	通过
压力模型	角度 0°、+13°、+15°	约 26、35、40 MPa	通过

前进后退	设置不同 MoveStep 并操作	转角随步长变化， 方向正确	通过
断开重连	关闭并重新打开串口	系统恢复通信	通过

表 7-1 系统功能测试结果

根据下位机 README 和进度记录，当前构建结果约为 FLASH 19152 B / 512 KB，RAM 2072 B / 64 KB，资源占用较低，说明系统仍有较大的扩展空间。实际测试表明，连续动作和快速点击不会造成同轴电机重复启动，busy 保护能够生效；上位机压力图表范围调整为 0 到 45 MPa 后，能够清晰显示正常、警告和紧急区间。

8 关键程序实现

本节列出系统中最能体现设计思路的关键程序段。完整工程文件已经保存在 **hardware** 和 **software** 目录中，报告正文选取下位机压力模型、前进后退目标位置控制、Modbus 应用映射、步进电机启动与停止、上位机输入寄存器读取和上位机报警判断等核心代码进行说明。这些代码分别对应数据模型、动作控制、通信协议、底层驱动和监控界面五个关键功能。

8.1 下位机压力模型关键代码

```
static void SensorSim_UpdatePressureFromAngle(void)
{
    if (angle_deg <= -13)
    {
        pressure1_x100 = SENSOR_PRESSURE_MIN_X100;
    }
    else if (angle_deg <= 0)
    {
        pressure1_x100 = SensorSim_DivRoundNearest((int32_t)(angle_deg + 13) *
                                                    SENSOR_PRESSURE_AT_ZERO_X100,
                                                    13);
    }
    else if (angle_deg < 13)
    {
        pressure1_x100 = SENSOR_PRESSURE_AT_ZERO_X100 +
            SensorSim_DivRoundNearest((int32_t)angle_deg *
            (SENSOR_PRESSURE_AT_EMERGENCY_START_X100 -
            SENSOR_PRESSURE_AT_ZERO_X100),
            13);
    }
    else if (angle_deg == 13)
    {
        pressure1_x100 = SENSOR_PRESSURE_AT_EMERGENCY_START_X100;
    }
    else if (angle_deg == 14)
    {
        pressure1_x100 = SENSOR_PRESSURE_AT_PLUS_14_X100;
    }
    else
    {
        pressure1_x100 = SENSOR_PRESSURE_MAX_X100;
    }

    pressure1_x100 = SensorSim_Clamp(pressure1_x100,
                                    SENSOR_PRESSURE_MIN_X100,
```

```

        SENSOR_PRESSURE_MAX_X100);
    pressure2_x100 = SensorSim_DivRoundNearest(pressure1_x100 *
        SENSOR_PRESSURE2_RATIO_NUMERATOR,
        SENSOR_PRESSURE2_RATIO_DENOMINATOR);
}

```

8.2 前进后退目标位置控制关键代码

```

void SupportMotion_MoveToPosition(uint16_t target_position)
{
    if (target_position > current_position)
    {
        uint16_t position_delta = (uint16_t)(target_position - current_position);

        if (SupportMotion_MoveByDelta(STEPPER_DIR_FORWARD, position_delta) == HAL_OK)
        {
            current_position = target_position;
            SensorSim_OnMoveForward();
        }
    }
    else if (target_position < current_position)
    {
        uint16_t position_delta = (uint16_t)(current_position - target_position);

        if (SupportMotion_MoveByDelta(STEPPER_DIR_BACKWARD, position_delta) ==
HAL_OK)
        {
            current_position = target_position;
            SensorSim_OnMoveBackward();
        }
    }
}

```

8.3 Modbus 寄存器与线圈映射关键代码

```

static ModbusException ModbusApp_ReadInputRegisters(uint16_t address,
        uint16_t quantity,
        uint16_t *values)
{
    uint16_t index;

    if ((quantity == 0u) ||
        (address >= MODBUS_INPUT_REG_COUNT) ||
        ((address + quantity) > MODBUS_INPUT_REG_COUNT))
    {
        return MODBUS_EXCEPTION_ILLEGAL_DATA_ADDRESS;
    }

    for (index = 0u; index < quantity; index++)
    {
        switch ((uint16_t)(address + index))
        {
            case MODBUS_INPUT_REG_ANGLE:

```

```

        values[index] = SensorSim_GetAngleRegister();
        break;

    case MODBUS_INPUT_REG_PRESSURE1:
        values[index] = SensorSim_GetPressure1Register();
        break;

    case MODBUS_INPUT_REG_PRESSURE2:
        values[index] = SensorSim_GetPressure2Register();
        break;

    default:
        return MODBUS_EXCEPTION_ILLEGAL_DATA_ADDRESS;
    }
}

return MODBUS_EXCEPTION_NONE;
}

static ModbusException ModbusApp_WriteSingleRegister(uint16_t address,
                                                    uint16_t value)
{
    if ((address == 0u) || (address >= MODBUS_HOLDING_REG_COUNT))
    {
        return MODBUS_EXCEPTION_ILLEGAL_DATA_ADDRESS;
    }

    holding_registers[address] = value;
    return MODBUS_EXCEPTION_NONE;
}

static ModbusException ModbusApp_WriteSingleCoil(uint16_t address,
                                                  uint8_t enabled)
{
    if (enabled == 0u)
    {
        return MODBUS_EXCEPTION_NONE;
    }

    switch (address)
    {
        case MODBUS_COIL_LOWER:
            SupportMotion_Lower();
            break;

        case MODBUS_COIL_RAISE:
            SupportMotion_Raise();
            break;
    }
}

```

```

        case MODBUS_COIL_MOVE:

SupportMotion_MoveToPosition(holding_registers[MODBUS_HOLDING_REG_TARGET_POSITION]);
        break;

        default:
            return MODBUS_EXCEPTION_ILLEGAL_DATA_ADDRESS;
    }

    return MODBUS_EXCEPTION_NONE;
}

```

8.4 步进电机启动与停止关键代码

```

HAL_StatusTypeDef Stepper_MoveSteps(StepperMotorId motor,
                                     StepperDirection direction,
                                     uint32_t steps,
                                     uint32_t frequency_hz)
{
    const BoardStepperConfig *config;
    HAL_StatusTypeDef status;

    if ((motor >= STEPPER_MOTOR_COUNT) || (steps == 0u))
    {
        return HAL_ERROR;
    }

    if (stepper_states[motor].busy != 0u)
    {
        return HAL_BUSY;
    }

    config = BoardConfig_GetStepper((BoardStepperId)motor);
    if (config == 0)
    {
        return HAL_ERROR;
    }

    status = Stepper_ApplyFrequency(config->timer, frequency_hz);
    if (status != HAL_OK)
    {
        return status;
    }

    stepper_states[motor].target_steps = steps;
    stepper_states[motor].current_steps = 0u;
    stepper_states[motor].busy = 1u;
}

```

```

    HAL_GPIO_WritePin(config->dir_port,
                      config->dir_pin,
                      Stepper_GetDirectionState(config, direction));
    HAL_GPIO_WritePin(config->ena_port, config->ena_pin, config->ena_active_state);

    status = HAL_TIM_PWM_Start_IT(config->timer, config->channel);
    if (status != HAL_OK)
    {
        stepper_states[motor].busy = 0u;
        HAL_GPIO_WritePin(config->ena_port,
                          config->ena_pin,
                          Stepper_GetInactiveEnaState(config->ena_active_state));
    }

    return status;
}

void Stepper_Stop(StepperMotorId motor)
{
    const BoardStepperConfig *config;

    if (motor >= STEPPER_MOTOR_COUNT)
    {
        return;
    }

    config = BoardConfig_GetStepper((BoardStepperId)motor);
    if (config == 0)
    {
        return;
    }

    HAL_TIM_PWM_Stop_IT(config->timer, config->channel);
    Stepper_ForceStepOutputIdle(config);
    HAL_GPIO_WritePin(config->ena_port,
                      config->ena_pin,
                      Stepper_GetInactiveEnaState(config->ena_active_state));
    stepper_states[motor].busy = 0u;

    if (Stepper_AnyBusy() == 0u)
    {
        active_frequency_hz = 0u;
    }
}

```

8.5 上位机传感器读取关键代码

```

Public Function ReadSupportSensors(slaveId As Byte) As (Angle As Double, Pressure1
As Double, Pressure2 As Double, IsSuccess As Boolean)
    If _master Is Nothing OrElse Not IsConnected Then

```

```

        Return (0, 0, 0, False)
    End If

    Try
        Dim registers As UShort()

        进入临界区，独占串口总线进行查询
        SyncLock _busLock
            ' 读取 3 个输入寄存器：起始地址 0，数量 3
            ' [0] 对应 角度，[1] 对应 压力 1，[2] 对应 压力 2
            registers = _master.ReadInputRegisters(slaveId, 0, 3)
        End SyncLock

        ' 【数据还原解析】：
        ' 单片机/PLC 通常无法直接传输浮点数（小数）。
        ' 这里的协议约定是：下位机把压力值放大了 100 倍变成整数传输，所以上位机拿到后必须除以 100.0 还原物理量。
        Dim angle As Double = CDb1(registers(0)) - 15.0
        Dim p1 As Double = registers(1) / 100.0
        Dim p2 As Double = registers(2) / 100.0

        Return (angle, p1, p2, True) ' 通信成功，返回 True

    Catch ex As Exception
        ' 发生任何超时或校验错误（CRC 不匹配等），均视为该次读取失败（设备可能离线、掉电或线路干扰）
        ' 此处不抛出异常，而是优雅地返回 False，让调用方（ViewModel）去更新界面的红灯离线状态

        Return (0, 0, 0, False)
    End Try
End Function

```

8.6 上位机报警判断关键代码

```

Private Sub CheckAlarms()
    If CurrentPressure1 >= 35.0 Then
        AddAlarm("紧急", $"压力过高 (超 35 MPa 阈值): {CurrentPressure1} MPa")
    ElseIf CurrentPressure1 >= 30.0 Then
        AddAlarm("警告", $"压力偏高 (超 30 MPa 阈值): {CurrentPressure1} MPa")
    End If
End Sub

```

9 总结与展望

本文围绕矿井液压支架监控对象，完成了一套结合虚拟仿真和硬件模拟仿真的上、下位机控制系统。硬件方面，系统以 STM32F103VET6 为核心，通过 GPIO 和 TIM2 PWM 控制 TB6600 步进驱动器，实现支架前进/后退和升高/降低的动作模拟；软件方面，下位机完成 Modbus RTU 从站、步进驱动、动作层和虚拟传感器模型，上位机完成 Modbus 主站、实时显示、报警和历史数据记录。系统从总体方案、硬件电路、软件结构、通信协议、测试调试到报告资料形成了完整闭环。

本设计的主要收获在于：第一，理解了 MCU 控制系统从 CubeMX 外设配置到应用层业务封装的完整过程；第二，掌握了 Modbus RTU 的帧结构、功能码、寄存器映射和主从轮询机制；第三，认识到上下位机状态同步在实际控制系统中的重要性，尤其是动作失败、忙碌保护和界面提前更新可能引发的逻辑偏差；第四，通过压力模型和角度边界设计，体会到仿真系统也需要保持物理含义和数据一致性。

系统仍有改进空间。后续可以将虚拟压力和角度传感器替换为真实传感器，接入压力变送器、倾角传感器或位移传感器；可以将串口 Modbus RTU 扩展为 RS485 多节点总线，真实验证 56 台支架轮询性能；可以补充紧急停止线圈、故障码寄存器、动作完成反馈和通信异常统计；也可以将上位机改造成更接近工业 SCADA 的界面，增加权限管理、数据导出、报警确认和历史报表。

9.1 目标达成分析

从专业综合设计角度看，本系统覆盖了方案论证、硬件系统设计、软件系统设计、系统调试和报告总结等主要环节。硬件部分不是停留在单片机最小系统说明，而是结合实际引脚分配、TB6600 驱动器接线、步进电机脉冲控制、电源供电方式和 PCB/BOM 材料进行说明；软件部分不是单一程序片段，而是形成了从 CubeMX 初始化、底层驱动、通信协议到应用层业务逻辑的完整结构。调试部

分记录了真实开发中遇到的问题和修正过程，包括前进后退目标位置差值、升降角度边界、压力模型同步和上位机显示适配。

从安全性和可靠性角度看，系统采用了多处防护措施。下位机对升降角度进行边界限制，避免显示角度到达极限后电机仍继续动作；步进驱动层采用 `busy` 状态保护，避免同轴电机在未完成当前动作时再次启动；移动控制采用目标位置差值逻辑，降低上下位机位置不同步造成的方向误判；通信层对 CRC 和从站地址进行校验，非法帧不会触发动作；上位机通过总线互斥锁管理轮询和控制命令，降低串口半双工冲突风险。这些措施虽然简单，但已经体现了工业控制系统设计中“先保证状态一致，再执行控制动作”的基本原则。

参考文献

- [1] STMicroelectronics. STM32F103xC, STM32F103xD, STM32F103xE high-density performance line Arm-based 32-bit MCU datasheet, DS5792 Rev 13, 2018.
- [2] Modbus Organization. MODBUS Application Protocol Specification V1.1b3, 2012.
- [3] Modbus Organization. MODBUS over Serial Line Specification and Implementation Guide V1.02, 2006.
- [4] Marios Tyrovolas, Tibor Hajnal. Inter-communication between Programmable Logic Controllers using IoT technologies: A Modbus RTU/MQTT Approach, 2021.
- [5] Christian Nikolajew, Holger Eichelberger. Industry 4.0 Connectors - A Performance Experiment with Modbus/TCP, 2024.
- [6] Microsoft. Windows Presentation Foundation documentation and .NET desktop development documentation.
- [7] LiveCharts Project. LiveCharts WPF charting library documentation.
- [8] 煤矿综采工作面液压支架监测与控制相关文献资料。
- [9] TB6600 步进电机驱动器产品资料。